Proyecto ConjuntoApp — Sistema de administración de propiedad horizontal

21 de octubre de 2026

El caso

ConjuntoApp es una aplicación web para administrar un conjunto residencial (propiedad horizontal) de Pereira con 4 torres y 120 apartamentos. La construyen 5 personas con MySQL, PHP y Angular, y se entrega el **jueves 19 de noviembre de 2026** (el 20 de noviembre de 2026 cae viernes; confirmen con el docente cuál es la fecha real).

Cliente simulado: Conjunto Residencial Los Guaduales, representado por su administradora.

Problema actual:

- Las reservas del salón social, BBQ y cancha se anotan en un cuaderno de portería, y hay choques de horario.
- Las PQRS (peticiones, quejas, reclamos y sugerencias) llegan por WhatsApp y se pierden; nadie mide tiempos de respuesta.
- Portería registra visitantes en papel; no hay control de parqueaderos de visitantes.
- Los residentes no conocen su estado de cuenta de cuotas de administración sin ir a la oficina.
- Los comunicados se pegan en carteleras que pocos leen.

Propuesta: un sistema web con roles que centraliza reservas, PQRS, portería, estado de cuenta de cuotas y comunicados, con un tablero de indicadores para la administración.

Objetivos y alcance

Objetivo general: desarrollar una aplicación web funcional, con base de datos relacional, que digitalice cinco procesos de la administración del conjunto.

Objetivos específicos:

1. Diseñar una base de datos MySQL normalizada (3FN) con integridad referencial.
2. Construir una API REST en PHP con autenticación por token y control de roles.
3. Construir un frontend en Angular con rutas protegidas por rol y formularios validados.
4. Entregar el sistema desplegado localmente, con datos de prueba y documentación.

Incluido
Login, roles y gestión de usuarios
Torres, apartamentos y residentes
Reservas de zonas comunes con validación de cruces
PQRS con estados, respuestas y adjuntos

Incluido
Portería: visitantes y parqueaderos de visitantes
Cuotas: generación mensual, registro manual de pagos, estado de cuenta
Comunicados y tablero de indicadores

Roles

El sistema maneja cuatro roles; cada pantalla y cada endpoint valida el rol del token.

Rol	Quién es	Qué puede hacer
Administrador	Administradora del conjunto	Todo: usuarios, apartamentos, cuotas, pagos, zonas, comunicados, responder PQRS, ver tablero
Residente	Propietario o arrendatario	Reservar zonas, radicar y seguir sus PQRS, ver su estado de cuenta, preautorizar visitantes, leer comunicados
Portero	Vigilante de turno	Registrar ingreso y salida de visitantes, asignar parqueadero de visitante, ver reservas del día
Consejo	Miembro del consejo de administración	Solo lectura: tablero, reportes de PQRS y cartera

Módulos y requisitos funcionales

Son cinco módulos, uno por integrante. Cada uno tiene CRUD, al menos una regla de negocio no trivial y una consulta con JOIN.

M1. Seguridad, usuarios y unidades

- RF-01 Iniciar y cerrar sesión con correo y contraseña (hash con `password_hash`).
- RF-02 CRUD de usuarios con asignación de rol; activar e inactivar.
- RF-03 CRUD de torres y apartamentos (número, piso, área m², coeficiente de copropiedad).
- RF-04 Asociar residentes a apartamentos (propietario o arrendatario, fecha de inicio y fin).
- RF-05 Cambiar contraseña y ver perfil.
- Regla: un apartamento tiene máximo un propietario activo; un usuario inactivo no puede iniciar sesión.

M2. Zonas comunes y reservas

- RF-06 CRUD de zonas comunes (nombre, aforo, horario de apertura y cierre, costo de uso, requiere depósito).
- RF-07 El residente consulta disponibilidad en un calendario y crea una reserva.
- RF-08 El administrador aprueba o rechaza reservas; el residente puede cancelar con 24 h de anticipación.
- RF-09 Vista del día para portería.
- Regla: no se permiten reservas cruzadas en la misma zona, fuera de horario, ni para apartamentos en mora de más de 2 cuotas.

M3. PQRS

- RF-10 El residente radica una PQRS (tipo, categoría, descripción, adjunto opcional).
- RF-11 Número de radicado automático (ej. PQRS-2026-0001).
- RF-12 El administrador cambia el estado (Radicada, En trámite, Respondida, Cerrada) y responde; queda historial.
- RF-13 El residente califica la respuesta de 1 a 5.
- RF-14 Reporte de PQRS por estado, categoría y tiempo promedio de respuesta.
- Regla: plazo de respuesta de 15 días hábiles; las vencidas se marcan en rojo.

M4. Portería y visitantes

- RF-15 El residente preautoriza un visitante (nombre, documento, fecha, con o sin vehículo).
- RF-16 El portero registra ingreso y salida, buscando por documento.
- RF-17 CRUD de parqueaderos de visitantes y asignación al ingresar.
- RF-18 Bitácora de ingresos filtrable por fecha, torre y apartamento.
- Regla: no se asigna un parqueadero ocupado; un visitante sin salida registrada aparece como “dentro”.

M5. Cuotas, pagos, comunicados y tablero

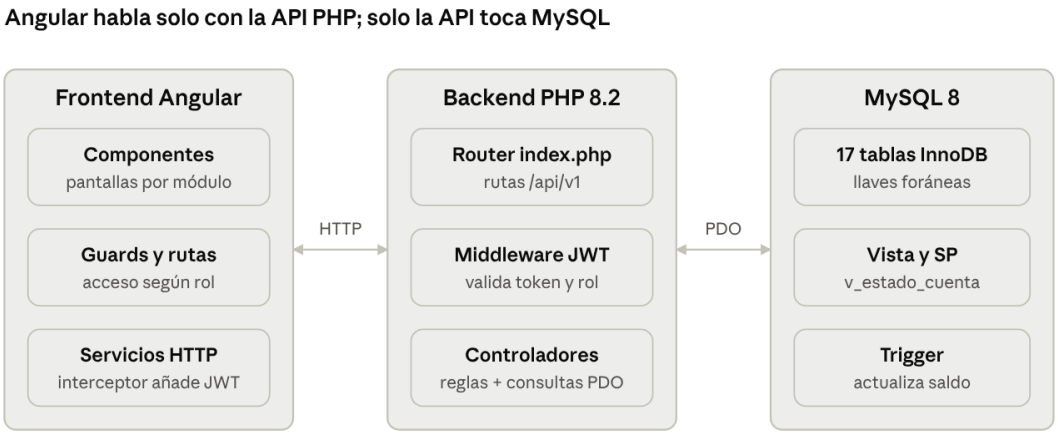
- RF-19 Generar las cuotas del mes para todos los apartamentos (valor según coeficiente).
- RF-20 Registrar pagos manuales (fecha, valor, medio, soporte) y aplicarlos a la cuota más antigua.
- RF-21 Estado de cuenta por apartamento con saldo y mora.
- RF-22 Publicar comunicados con fecha de vigencia y marcar como leídos.
- RF-23 Tablero: % de cartera, PQRS abiertas, reservas de la semana, visitantes del día.
- Regla: una cuota no puede generarse dos veces para el mismo apartamento y periodo.

Requisitos no funcionales

- **Seguridad:** contraseñas con `password_hash` (bcrypt); JWT con expiración de 8 h; validación de rol en cada endpoint; CORS limitado al origen del frontend.
- **Usabilidad:** interfaz responsive (escritorio y celular), mensajes de error en español, confirmación antes de eliminar.
- **Rendimiento:** listados paginados de 20 registros; respuesta de la API menor a 1 s con los datos de prueba.
- **Integridad:** llaves foráneas, UNIQUE donde aplique, borrado lógico (`activo`) en catálogos.
- **Mantenibilidad:** estructura por capas, nombres en español

Arquitectura y stack técnico

Es una arquitectura cliente-servidor de tres capas: Angular consume una API REST en PHP, y solo la API accede a MySQL.



Arquitectura · 3 capas

Cada petición pasa por el router, luego por el middleware que valida el token y el rol, y llega al controlador del módulo.

Capa	Tecnología	Detalle
Frontend	Angular (versión estable actual, componentes standalone), TypeScript, Bootstrap 5 o Angular Material	Reactive Forms, guards por rol, carga diferida por módulo
Backend	PHP 8.2 sin framework (o Slim 4 si el docente lo permite), Composer	No usar PDO
Base de datos	MySQL 8	InnoDB, utf8mb4, scripts versionados en el repositorio
Herramientas	Git y GitHub, Postman, XAMPP, MySQL, GitHub Projects o Trello	—

Modelo de datos (MySQL 8, InnoDB, utf8mb4)

Son 17 tablas. Es la propuesta base: el equipo la valida en la semana 1 y la congela antes de programar.

Tabla	Campos principales	Relaciones
roles	id, nombre	—
usuarios	id, rol_id, nombres, apellidos, documento (UNIQUE), correo (UNIQUE), telefono, password_hash, activo	roles
torres	id, nombre, pisos	—
apartamentos	id, torre_id, numero, piso, area_m2, coeficiente	torres; UNIQUE (torre_id, numero)
residencias	id, usuario_id, apartamento_id, tipo (propietario/arrendatario), fecha_inicio, fecha_fin	usuarios, apartamentos
zonas_comunes	id, nombre, aforo, hora_apertura, hora_cierre, costo, requiere_deposito, activo	—
reservas	id, zona_id, apartamento_id, usuario_id, fecha, hora_inicio, hora_fin, estado, observaciones	zonas_comunes, apartamentos, usuarios
categorias_pqrs	id, nombre	—
pqrs	id, radicado (UNIQUE), tipo, categoria_id, usuario_id, apartamento_id, asunto, descripcion, adjunto, estado, fecha_limite, calificacion	categorias_pqrs, usuarios, apartamentos
pqrs_seguimiento	id, pqrs_id, usuario_id, estado_anterior, estado_nuevo, comentario, fecha	pqrs, usuarios
visitantes	id, documento (UNIQUE), nombre, telefono	—
autorizaciones	id, visitante_id, apartamento_id, usuario_id, fecha_visita, con_vehiculo, placa	visitantes, apartamentos, usuarios
parqueaderos_visitantes	id, codigo, ocupado	—
ingresos	id, visitante_id, apartamento_id, portero_id, parqueadero_id (NULL), placa, fecha_ingreso, fecha_salida	visitantes, apartamentos, usuarios, parqueaderos_visitantes
cuotas	id, apartamento_id, periodo (AAAA-MM), valor, fecha_vencimiento, saldo, estado	apartamentos; UNIQUE (apartamento_id, periodo)
pagos	id, cuota_id, valor, fecha_pago, medio, soporte, registrado_por	cuotas, usuarios
comunicados	id, titulo, contenido, fecha_publicacion, vigente_hasta, autor_id	usuarios

Opcional: `comunicados_leidos (comunicado_id, usuario_id, fecha)` para marcar lectura.

Mínimo esperado en la BD: un script `schema.sql`, un `seed.sql` con datos de prueba (4 torres, 120 apartamentos, 30 usuarios, 3 meses de cuotas), al menos 1 vista (`v_estado_cuenta`), 1 procedimiento almacenado (`sp_generar_cuotas`) y 1 trigger (actualizar `saldo` al insertar un pago).

Distribución del equipo

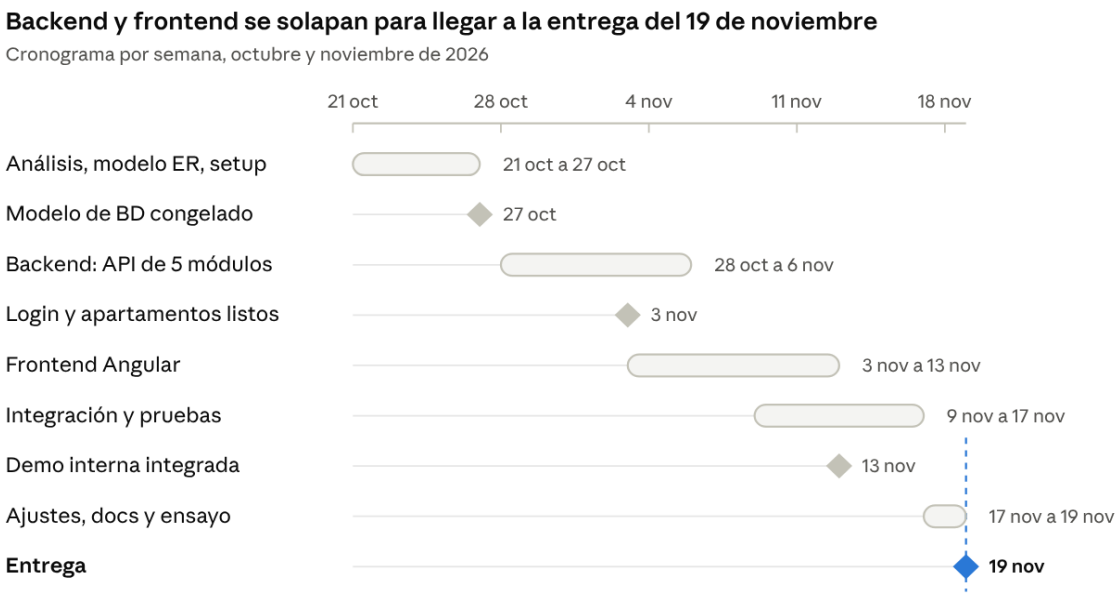
Cada integrante es dueño de un módulo de punta a punta (tablas, endpoints PHP, pantallas Angular y pruebas) y además tiene un rol transversal. Así todos tocan las tres capas y nadie queda bloqueado esperando a otro.

Integrante	Módulo (full stack)	Rol transversal
Integrante 1	M1 Seguridad, usuarios y unidades	Líder técnico: estructura del backend, JWT, middleware de roles, revisa pull requests
Integrante 2	M2 Zonas comunes y reservas	Líder de frontend: proyecto Angular, layout, guards, interceptor, servicios base
Integrante 3	M3 PQRS	Responsable de BD: modelo ER, `schema.sql`, `seed.sql`, vista, procedimiento y trigger
Integrante 4	M4 Portería y visitantes	QA: plan de pruebas, colección de Postman, registro de bugs
Integrante 5	M5 Cuotas, comunicados y tablero	Scrum master y documentación: tablero de tareas, actas, manual de usuario, presentación

M1 va primero porque los demás dependen del login y de los apartamentos: el integrante 1 entrega `POST /auth/login` y el CRUD de apartamentos al final de la semana 2.

Cronograma

Son 4 semanas de trabajo más 2 días de cierre. Los festivos del lunes 2 y lunes 16 de noviembre ya están descontados.



Cronograma · 21 oct a 19 nov 2026

Qué debe estar listo en cada corte:

1. **27 oct:** requisitos validados, modelo ER y `schema.sql` congelados, repositorio creado, proyecto Angular y API PHP "hola mundo" corriendo en las 5 máquinas.
2. **3 nov:** login con JWT, middleware de roles y CRUD de apartamentos funcionando; `seed.sql` con datos de prueba.
3. **6 nov:** endpoints de los 5 módulos probados en Postman.
4. **13 nov:** pantallas de los 5 módulos conectadas a la API; demo interna completa por rol.
5. **17 nov:** bugs críticos cerrados, tablero, reportes y manual de usuario listos.
6. **19 nov:** entrega final y presentación.

Un requisito está terminado cuando: funciona en la BD de prueba, valida datos en frontend y backend, respeta el rol, está en la rama `develop` por pull request revisado por otra persona y tiene su caso en Postman.

Forma de trabajo: ramas `main` (entregable), `develop` (integración) y `feature/m2-reservas` por tarea; commits pequeños con mensaje claro; reunión de 15 minutos lunes, miércoles y viernes.

Riesgo	Mitigación
El login se atrasa y bloquea a todos	Hasta tenerlo, el frontend usa un token simulado y el backend un usuario fijo
Cambios en la BD rompen módulos ajenos	Modelo congelado el 27 de octubre; cambios solo con migración ALTER acordada
Integración tardía revela errores	Integrar en `develop` cada semana, no al final
Diferencias de entorno (versiones de PHP o MySQL)	Todos con XAMPP o Laragon con PHP 8.2 y MySQL 8; `.env` de ejemplo en el repo

Riesgo	Mitigación
Festivos del 2 y 16 de noviembre	Ya descontados en el cronograma; usar esos días como reserva
Un integrante se retira o se atrasa	Cada módulo documentado en el README para que otro lo retome